Sentinel: In-Database Real-Time Fraud & Anomaly Engine

Course: CS/AI/AIM/DS 202 Database Management Systems (Semester III, A.Y. 2026-27)
Institution: ICFAI Foundation for Higher Education (IcfaiTech), Hyderabad **Authors:** Sai Haresh Anand S (25STUCHH010419) and Dulam Seshivardhini (25STUCHH001048)
Platform: MySQL 8.4 (Docker), pure SQL; no application code in the detection path

---

1. Abstract

Fraud checks are normally written in the application that accepts a payment. That arrangement has a structural weakness: any code path that writes to the database without going through the check, whether a batch job, an admin script or a second service, produces a transaction nobody scored. Sentinel moves the check into the database itself. Every `INSERT` into `transactions` fires an `AFTER INSERT` trigger that calls the stored procedure `score_transaction()`. The procedure evaluates six rules (amount Z-score, impossible travel by the Haversine formula, frequency velocity, unseen device, off-hours activity and merchant risk) against thresholds stored in the `rule_config` table, and writes a row to `alerts` with a JSON array explaining every rule that fired.

The schema has 13 tables in third normal form, 11 views, a user-defined function, four procedures, three triggers, a nightly event, four roles and a seeded data set of 12,067 transactions across 500 users and 597 accounts. Fund transfers run through `transfer_funds()`, which uses explicit row locks and a fixed lock order to stay atomic and deadlock-free. Measured on a laptop, the trigger raises the cost of an insert from about 0.04 ms to about 2 ms, which still sustains roughly 500 transactions per second. A date-range report falls from a full scan (cost 1,349) to an index range scan (cost 174) after a single-column index on `txn_time`. The project is verified by 102 SQL assertions and 27 Python integration tests.

2. Problem Statement and Objectives

2.1 Limitations of application-tier fraud checks

1. **Bypass.** The check runs only where a developer remembered to call it. Direct SQL, imports and a second service skip it.
2. **Race window.** If the application commits first and scores afterwards, there is an interval in which an unscored transaction is visible and spendable.
3. **Data movement.** Scoring needs history (an account's mean and deviation, its previous location, the last five minutes of activity). Pulling that history across the network for every payment adds latency and load, while the database already holds it next to an index.

4. **Hard-coded policy.** Thresholds live in source code, so changing one means a deployment.
5. **No audit trail by default.** Why an alert fired is usually a log line, not queryable data.

2.2 Objectives

#	Objective	How Sentinel meets it
O1	Score every transaction regardless of who inserted it	`AFTER INSERT` trigger on `transactions`
O2	Explain every alert	`alerts.reasons` JSON array, one entry per rule fired
O3	Make policy data, not code	`rule_config` holds every threshold and an enabled flag
O4	Keep money movement all-or-nothing	`transfer_funds()` with `START TRANSACTION`, `FOR UPDATE`, `COMMIT/ROLLBACK`
O5	Least-privilege access	Four roles; the reporting role cannot read raw transactions
O6	Measure the cost honestly	`EXPLAIN ANALYZE` plans and timed inserts, with limitations documented

3. Requirements and Entity-Relationship Design

3.1 Functional requirements

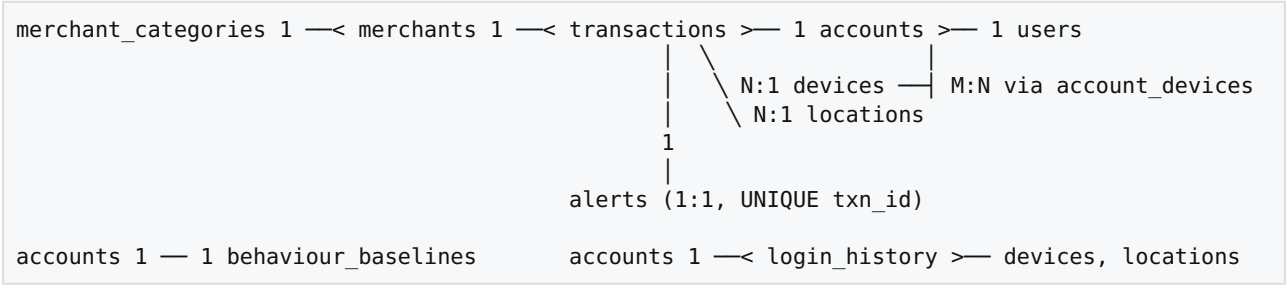
- Store users, their accounts, the devices and locations they transact from, and the merchants they pay.
- Record each transaction with amount, time, merchant, device, location and status.
- On each new transaction, decide whether it is suspicious and record why.
- Let an analyst move an alert through `open → investigated → confirmed | false_positive`, with the review time stamped and the change audited.
- Provide reporting views for dashboards without exposing base tables.
- Move funds between accounts atomically.

3.2 Entity sets

Table	Role	Primary key
`merchant_categories`	Lookup: category names	`category_id`
`locations`	Lookup: city, state, country, latitude, longitude	`location_id`
`devices`	Device type, OS, browser, fingerprint hash	`device_id`
`rule_config`	One row per fraud rule: enabled flag, threshold, parameters JSON	`rule_id`
`users`	Customer identity (name, unique email, phone)	`user_id`
`accounts`	Savings or current account owned by a user, with balance and status	`account_id`
`merchants`	Merchant with category, home coordinates, risk score	`merchant_id`
`account_devices`	Associative table for the many-to-many link between accounts and devices	(`account_id`, `device_id`)

Table	Role	Primary key
transactions	The fact table; 12,067 rows	txn_id
alerts	Output of the scoring engine	alert_id
behaviour_baselines	Per-account average, deviation, usual hours and cities	baseline_id
login_history	Account logins by device and location	login_id
audit_log	Who changed which row, with old and new values as JSON	audit_id

3.3 Relationships and foreign-key mapping



Foreign key	References	On delete	Reasoning
accounts.user_id	users	CASCADE	An account has no meaning without its owner
merchants.category_id	merchant_categories	RESTRICT	A category in use cannot disappear
account_devices.account_id / device_id	accounts / devices	CASCADE	A link row dies with either side
transactions.account_id	accounts	CASCADE	History belongs to the account
transactions.merchant_id / device_id / location_id	merchants / devices / locations	RESTRICT	Referenced reference data must not vanish silently
alerts.txn_id	transactions	CASCADE	An alert explains one transaction
behaviour_baselines.account_id	accounts	CASCADE	Derived data follows its account
login_history.account_id	accounts	CASCADE	
login_history.device_id / location_id	devices / locations	RESTRICT	

Other declared constraints: UNIQUE on users.email, devices.fingerprint_hash, rule_config.rule_name, merchant_categories.category_name, (locations.city, locations.country), alerts.txn_id and behaviour_baselines.account_id; CHECK (amount > 0) on transactions; CHECK (balance >= 0) on accounts; range checks on latitude and longitude.

Design note. The schema was built week by week: Week 2 creates bare tables and Week 3 adds keys and constraints with ALTER TABLE. The result is identical to declaring everything inline; a schema fingerprint script compares it to a reference. The alerts table is created as fraud_alerts in Week 2 and renamed to alerts with its stray notes column dropped, which is why the design documents and the live database say alerts.

4. Normalised Relational Schema (1NF, 2NF, 3NF)

Method. For each table, list the functional dependencies (FDs) that hold in the business rules, determine the candidate keys, and check three conditions.

- **1NF:** every column is atomic and single-valued, and every row is identified by a key.
- **2NF:** no non-key attribute depends on a proper part of a composite key.
- **3NF:** for every non-trivial FD X → A, either X is a superkey or A is part of a candidate key. Equivalently, no non-key attribute depends on another non-key attribute.

4.1 Table-by-table verification

Table	Candidate keys	Non-trivial functional dependencies	Result
`users`	`user_id`, `email`	`user_id → full_name, email, phone, created_at`; `email → user_id`	Every determinant is a candidate key → **3NF (BCNF)**
`accounts`	`account_id`	`account_id → user_id, account_type, balance, status, created_at`	Single-column key rules out partial dependency (2NF). No non-key column determines another → **3NF**
`merchants`	`merchant_id`	`merchant_id → merchant_name, category_id, city, latitude, longitude, risk_score`	The category name is not stored here; it is reached through `category_id`, so `category_id → category_name` stays in `merchant_categories` → **3NF**
`transactions`	`txn_id`	`txn_id → account_id, merchant_id, device_id, location_id, amount, txn_time, status, metadata_json`	City, device type and merchant name are not repeated here; they live in the tables the foreign keys point to → **3NF**
`rule_config`	`rule_id`, `rule_name`	`rule_id → rule_name, enabled, threshold_value, parameters_json, updated_at`; `rule_name → rule_id`	Both determinants are candidate keys → **3NF (BCNF)**
`alerts`	`alert_id`, `txn_id`	`alert_id → txn_id, rule_name, risk_score, reasons, status, created_at, reviewed_at`; `txn_id → alert_id`	Both determinants are candidate keys → **3NF (BCNF)**
`account_devices`	(`account_id`, `device_id`)	(`account_id`, `device_id`) → `first_seen`, `last_seen`	The only composite-key table. Both columns describe the pairing, not either side alone, so there is no partial dependency → **2NF and 3NF**

The remaining six tables (`merchant_categories`, `locations`, `devices`, `behaviour_baselines`, `login_history`, `audit_log`) each have a single-column surrogate key and no dependency between non-key columns. In `locations`, `city → country` is not declared as a dependency because the same city name can occur in more than one country; the candidate key is (`city`, `country`) and `latitude`, `longitude` depend on it.

4.2 1NF in the presence of JSON columns

`alerts.reasons`, `rule_config.parameters_json`, `transactions.metadata_json` and the two JSON columns in `behaviour_baselines` hold structured documents. They are treated as atomic values: the engine never joins on or filters by an element inside them. `reasons` is output meant to be read whole by an analyst, and `parameters_json` carries free-form rule parameters. If the project later needed to query "all alerts that cite rule X", that would be the signal to extract a child table.

4.3 The deliberate exception

`behaviour_baselines` stores values that can be computed from `transactions` (mean, standard deviation, active hours, typical cities). It is a controlled cache rather than a normalisation error: the nightly procedure rebuilds it in full, so it cannot drift for more than a day, and it saves recomputing aggregates over an account's history for reporting.

5. Advanced Database Features

5.1 The six fraud rules

Each rule adds one point to the score and appends a human-readable string to the `reasons` array. A transaction is alerted if at least one rule fires. There is no combined threshold; each rule has its own, read from `rule_config` at scoring time.

#	Rule	Condition	Default threshold
1	**Amount Z-score**	`(amount − AVG) / STDDEV_SAMP` over the account's history exceeds the threshold; requires at least 5 prior-or-current rows and a non-zero deviation	3
2	**Travel velocity**	Haversine distance to the previous transaction is above 50 km and implied speed exceeds the limit	900 km/h
3	**Frequency velocity**	Number of transactions by the account inside the window, including the new one, exceeds the limit	more than 5 in 5 minutes
4	**New device**	The device is not in `account_devices` for this account	1 (enabled)
5	**Off-hours**	The hour of day accounts for less than 2% of the account's history	0.02
6	**Merchant risk**	The merchant has at least 3 transactions and at least 3 alerts, and its alert rate divided by the platform-wide alert rate exceeds the threshold	2 (min. alerts 3)

Haversine. Implemented as the deterministic function `haversine_km(lat1, lon1, lat2, lon2)` with Earth radius R = 6371 km:

```
a = sin²(Δφ/2) + cos φ1 · cos φ2 · sin²(Δλ/2)
d = 2R · asin(√a)
```

Hyderabad (17.385, 78.4867) to Tokyo (35.6762, 139.6503) evaluates to 6,311.39 km in MySQL. Fifteen minutes apart, that is an implied 25,246 km/h against a 900 km/h limit.

The 50 km floor stops two nearby cities with close timestamps from producing a spurious huge speed from small distances and short intervals.

A property of the Z-score rule worth stating. Because the trigger fires after the insert, the new row is already inside the average and standard deviation it is compared to. For a sample of n values, a single outlier cannot exceed a Z of $(n - 1)/\sqrt{n}$. With 65 transactions that ceiling is about 7.9. In practice this makes the rule slightly conservative for very small histories, and it is documented as a limitation rather than corrected, because computing the baseline from earlier rows only would require a second query per insert.

5.2 Scoring pipeline: `trg_after_txn_insert` and `score_transaction()`

```
CREATE TRIGGER trg_after_txn_insert
AFTER INSERT ON transactions
FOR EACH ROW
BEGIN
    CALL score_transaction(NEW.txn_id);
END;
```

The trigger is `AFTER`, not `BEFORE`, because the procedure reads the stored row and a `BEFORE` trigger runs before the row exists. It is row-level, so a multi-row `INSERT` scores every row.

`score_transaction(p_txn_id)` loads the transaction and the matching `rule_config` rows, evaluates each enabled rule, builds the `reasons` array with `JSON_ARRAY` and, when the score is above zero, writes

```
INSERT INTO alerts (txn_id, rule_name, risk_score, reasons, status)
VALUES (..., 'combined', v_score, v_reasons, 'open')
ON DUPLICATE KEY UPDATE ...;
```

The `UNIQUE (txn_id)` key makes re-scoring idempotent. When the score is zero, any existing open alert for that transaction is deleted, so re-running the procedure after a threshold change leaves the table consistent. A bulk `backfill_alerts(first, last)` procedure applies the same logic to historical ranges.

A second pair of triggers handles the analyst workflow. `trg_before_alert_update` stamps `reviewed_at` when an alert leaves `open`. `trg_after_alert_update` writes the old and new status to `audit_log`, using `USER()` rather than `CURRENT_USER()`: inside a trigger `CURRENT_USER()` returns the definer, so every audit row would name root.

5.3 Baseline maintenance: `refresh_behaviour_baselines()`

```
CREATE PROCEDURE refresh_behaviour_baselines()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
    START TRANSACTION;
    DELETE FROM behaviour_baselines;
    INSERT INTO behaviour_baselines (account_id, avg_amount, stddev_amount,
                                     peak_hours_json, typical_cities_json)
    SELECT ... AVG(amount), STDDEV_SAMP(amount), JSON_ARRAYAGG(hour), JSON_ARRAYAGG(city)
    FROM transactions ... GROUP BY account_id;
```

```
    COMMIT;
END;
```

The delete and rebuild sit in one transaction with an exit handler, so a failure leaves the previous baselines intact instead of an empty table. The event `evt_refresh_baselines` calls it every day at 02:30 (the event scheduler must be on).

5.4 ACID transfers: `transfer_funds()`

The procedure moves money between two accounts and records the debit as a transaction, which the scoring trigger then evaluates like any other payment.

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;

IF p_amount IS NULL OR p_amount <= 0 THEN SIGNAL SQLSTATE '45000' ...;
IF p_from_account = p_to_account      THEN SIGNAL SQLSTATE '45000' ...;

START TRANSACTION;

IF p_from_account < p_to_account THEN
    SELECT balance, status INTO v_from_balance, v_from_status
      FROM accounts WHERE account_id = p_from_account FOR UPDATE;
    SELECT status INTO v_to_status
      FROM accounts WHERE account_id = p_to_account   FOR UPDATE;
ELSE
    SELECT status INTO v_to_status
      FROM accounts WHERE account_id = p_to_account   FOR UPDATE;
    SELECT balance, status INTO v_from_balance, v_from_status
      FROM accounts WHERE account_id = p_from_account FOR UPDATE;
END IF;

-- missing / inactive account or insufficient funds: ROLLBACK, then SIGNAL

UPDATE accounts SET balance = balance - p_amount WHERE account_id = p_from_account;
UPDATE accounts SET balance = balance + p_amount WHERE account_id = p_to_account;
INSERT INTO transactions (account_id, merchant_id, device_id, location_id, amount, status)
VALUES (p_from_account, 1, p_device_id, p_location_id, p_amount, 'success');

COMMIT;
```

Property	Mechanism
Atomicity	Debit, credit and the transaction record commit together; the exit handler rolls back on any error, including one raised inside the scoring trigger
Consistency	Validation in the procedure, plus `CHECK (balance >= 0)` as a backstop
Isolation	`SELECT … FOR UPDATE` takes exclusive row locks, so a concurrent transfer on the same account waits instead of reading a stale balance
Durability	InnoDB redo log flushed at `COMMIT`

Deadlock avoidance. If transfer A→B and transfer B→A each locked their own source first, each would then wait for the other's lock forever. The procedure always locks the lower `account_id` first. All transfers acquire locks in the same global order, so a cycle of waits cannot form.

Scope limit. Only the sender's debit is recorded as a transaction, against placeholder merchant 1. Money received is not a purchase and would distort the receiver's spending statistics.

5.5 Indexing and performance

Indexes were created in Week 9 after the schema and the rule queries existed, so each one corresponds to a measured access path:

Index	Serves
`idx_txn_account_time (account_id, txn_time)`	Per-account velocity, previous-transaction lookup, baselines, latest-N queries
`idx_txn_merchant_time (merchant_id, txn_time)`	Merchant risk and merchant timelines
`idx_txn_time (txn_time)`	Date-range reports
`idx_txn_location (location_id)`	Location joins
`idx_alert_status (status, created_at)`	The open-alerts view
`idx_alert_rule (rule_name)`	Alerts by rule

Measured with `EXPLAIN ANALYZE` (MySQL 8.4.11).

Query	Before	After	Plan change
Daily report over a 6-day range (reads of 12,067 rows → 315)	cost **1,349**, table scan	cost **174**, range scan on `idx_txn_time`	about 8× cheaper
Latest 50 transactions of one account	cost 18.2 with filesort	cost 18.2, no sort	Same cost; the composite index removes the `Sort` step because rows come out already ordered
Merchant 50, newest 20	cost 88.5	reverse scan on `idx_txn_merchant_time`	no sort

The reduction from 1,349 to 174 therefore comes from the single-column index on `txn_time` serving a range predicate. The composite index on `(account_id, txn_time)` matters for a different reason: an account holds only about 52 rows on average, so the optimiser's cost does not drop, but the index lets the velocity and travel rules find an account's recent rows in order without sorting. We report both results as measured, and do not credit the composite index with the larger figure.

6. Experimental Benchmarks and Evaluation

Environment. MySQL 8.4.11 in Docker on a laptop under WSL2; single connection; data set of 12,067 transactions (500 users, 597 accounts, 50 merchants).

6.1 Trigger overhead

1,000 inserts were timed inside a transaction that was then rolled back, once with the trigger dropped and once with it present.

Configuration	Time for 1,000 inserts	Per row
No trigger	about 41 ms	about 0.04 ms
With `trg_after_txn_insert`	about 1,980 ms	about 2 ms

Scoring makes each insert roughly 50× slower in relative terms, and about 2 ms in absolute terms. That corresponds to approximately 500 scored transactions per second on one connection. For a payment path where a person is waiting on a card or UPI confirmation, 2 ms is negligible; for bulk loading it is not.

6.2 Bulk loading

Loading seed data with the trigger already installed stalled at 5,925 rows after 250 seconds, because every row ran the full scoring procedure against a growing table. The repository therefore loads data first and creates the trigger last (about 20 s for weeks 1–9), then uses `backfill_alerts()` to score history in one pass: `backfill_alerts(1, 12067)` takes about 38 s and produces **634 alerts, about 5%** of transactions.

6.3 Detection check

Twenty-seven planted persona transactions (ids 12041–12067) exercise each rule. They produce 21 alerts, and an ordinary purchase produces none. The database-side checks (102 assertions across 11 weeks) and the Python tests (27 tests) all pass at the time of writing.

6.4 Known limitations

- The seed data assigns random cities to transactions, so the travel rule fires 32–46 times on data that is synthetic rather than fraudulent.
- On amounts with a skewed distribution, a Z-score above 3 flags about 1.2% of rows (145 of 12,067). Amount is a poor fit for a normal-distribution test; a log transform would help.
- The off-hours rule is unreliable below about 50 transactions per account.
- Merchant risk feeds on alerts that other rules produce, so it can reinforce itself. The `min_alerts = 3` guard limits this but does not remove it.
- The baseline includes the row being scored (see 5.1).
- There is no user interface, and the code relies on MySQL-specific syntax.

7. Viva Questions and Discussion

Q1. What does the scoring trigger cost, and is it acceptable? It adds about 2 ms per inserted row in our measurement (about 0.04 ms without it), so a single connection scores about 500 transactions per second. For interactive payments that is far below human-visible latency, and the work is a few indexed lookups. It becomes a problem for bulk loads: loading our seed with the trigger present stalled, so we load first, create the trigger afterwards and use `backfill_alerts()` for history. Past a few thousand scored inserts per second one would move scoring to an asynchronous queue, which gives up the guarantee that scoring completes before the insert returns.

Q2. Why is the trigger `AFTER INSERT` and not `BEFORE`? `score_transaction()` looks the transaction up by id and aggregates over the account's rows including this one, so the row has to exist. A `BEFORE` trigger could modify `NEW` or reject the row, but it runs before the

row is stored. We use `BEFORE UPDATE` only where changing `NEW` is the point, to stamp `reviewed_at`. The consequence of `AFTER` is that the new row participates in the average and deviation, which caps the achievable Z-score near (n−1)/√n.

Q3. Explain the locking in `transfer_funds()`. How do you avoid deadlock? `SELECT … FOR UPDATE` inside a transaction takes an exclusive InnoDB row lock on the account, held until `COMMIT` or `ROLLBACK`. A second transfer touching the same account blocks, and when it resumes it reads the committed balance, so two transfers cannot both spend the same money. Deadlock occurs when two sessions hold one lock each and want the other's. We always lock the lower account id first, so A→B and B→A both take the same row first and the second simply waits. InnoDB would otherwise detect the cycle and abort one transaction; the lock order means that never has to happen.

Q4. What is the trade-off of keeping fraud logic in the database rather than in the application? Gains: it cannot be bypassed by any writer, there is no gap between commit and check, history is read next to its indexes, and thresholds are rows that an administrator can change without a release. Costs: the work consumes database CPU on the write path, the logic is MySQL-specific and harder to unit-test and debug than application code, a bug in a trigger can fail the originating insert, and scaling the database (usually the harder tier to scale) now also scales fraud scoring. Our compromise is to keep rules simple, indexed and set-based, and to keep model-style analysis outside the database.

Q5. Your report shows cost falling from 1,349 to 174. What caused it, and what does the composite index do? The 1,349 → 174 reduction is a date-range report on `transactions` moving from a full table scan of 12,067 rows to an index range scan of 315 rows on the single-column index `idx_txn_time`. The composite index `(account_id, txn_time)` follows the leftmost-prefix rule: it supports filters on `account_id` or on `account_id` plus a time range, and it returns an account's rows already sorted by time. In our measurement it did not lower the optimiser's cost for "latest 50 of an account" (18.2 either way, as an account has only about 52 rows) but it removed the sort step, which matters for the scoring rules. It does not help a filter on `txn_time` alone, which is why both indexes exist.

8. Conclusion and Future Scope

Sentinel shows that a useful fraud check can live entirely inside a relational database. A single trigger guarantees that every transaction is scored, whoever inserted it; thresholds are ordinary rows; every alert carries a machine-readable explanation; and money transfers keep ACID guarantees under concurrent access through a fixed lock order. The measured cost is about 2 ms per insert, and indexing the access paths reduced a representative report from cost 1,349 to 174. Equally important, the evaluation records where the design is weak: synthetic geography inflates the travel rule, the Z-score is a poor fit for skewed amounts, and the baseline includes the row under test.

Future work.

1. Calculate the baseline from earlier transactions only, removing the Z-score ceiling.

2. Assign each account a home city, so the travel rule compares against habitual locations and not just the previous row.
3. Score merchant risk from confirmed fraud (`alerts.status = 'confirmed'`) instead of from raw alerts, to break the feedback loop.
4. Replace the plain Z-score with a robust measure (median and MAD, or a log-amount Z-score).
5. Add a thin read-only API on the `api_reader` role and a dashboard on top of the existing reporting views.
6. Add a `txn_type` column so transfers, purchases and refunds are scored differently.
7. Test the asynchronous alternative (queue plus worker) to quantify the latency/throughput trade-off against the in-trigger design.

Figures in this report were measured on the project database; the supporting scripts and EXPLAIN output are in `week-09-views-indexes-dcl/docs/benchmark_measured.md` and `week-11-final-polish/docs/performance.md`.

Appendix A. Source Listings

Every listing below is the file as committed in the repository; nothing has been abridged except where stated.

A.1 Constraints (week 03)

`week-03-constraints/sql/01_constraints.sql`

```
-- ============================================================================
-- Week 3 — Constraints: PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT, CHECK, FOREIGN KEY
-- 01_constraints.sql — turn the bare Week 2 tables into a protected schema
-- ============================================================================
-- Order matters:  keys first (FOREIGN KEYs need a PRIMARY KEY or UNIQUE key to point at),
-- foreign keys LAST (the parent tables must already have their keys).
-- "NULL" from the lab manual's list is the default state of a column: any column that never
-- gets NOT NULL below (phone, description, state, metadata_json, reviewed_at ...) is
deliberately optional.
--
-- Every statement is an ALTER TABLE on a table created in Week 2.

USE sentinel;

-- ============================================================================
-- 1. PRIMARY KEY  (+ AUTO_INCREMENT so the database hands out the ids)
-- ============================================================================
ALTER TABLE merchant_categories MODIFY category_id INT    AUTO_INCREMENT PRIMARY KEY;
ALTER TABLE locations           MODIFY location_id INT    AUTO_INCREMENT PRIMARY KEY;
ALTER TABLE devices             MODIFY device_id   INT    AUTO_INCREMENT PRIMARY KEY;
ALTER TABLE rule_config         MODIFY rule_id     INT    AUTO_INCREMENT PRIMARY KEY;
ALTER TABLE users               MODIFY user_id     INT    AUTO_INCREMENT PRIMARY KEY;
ALTER TABLE accounts            MODIFY account_id  INT    AUTO_INCREMENT PRIMARY KEY;
ALTER TABLE merchants           MODIFY merchant_id INT    AUTO_INCREMENT PRIMARY KEY;
ALTER TABLE transactions        MODIFY txn_id      BIGINT AUTO_INCREMENT PRIMARY KEY;
ALTER TABLE alerts              MODIFY alert_id    BIGINT AUTO_INCREMENT PRIMARY KEY;
ALTER TABLE behaviour_baselines MODIFY baseline_id INT    AUTO_INCREMENT PRIMARY KEY;
ALTER TABLE login_history       MODIFY login_id    BIGINT AUTO_INCREMENT PRIMARY KEY;
ALTER TABLE audit_log           MODIFY audit_id    BIGINT AUTO_INCREMENT PRIMARY KEY;
```

```
-- Composite primary key: one row per (account, device) pair. This is what makes the M:N table
work.
ALTER TABLE account_devices ADD PRIMARY KEY (account_id, device_id);

-- ============================================================================
-- 2. NOT NULL
-- ============================================================================
ALTER TABLE merchant_categories MODIFY category_name VARCHAR(100) NOT NULL;

ALTER TABLE locations
    MODIFY city      VARCHAR(100)  NOT NULL,
    MODIFY country   VARCHAR(100)  NOT NULL,
    MODIFY latitude  DECIMAL(9,6)  NOT NULL,
    MODIFY longitude DECIMAL(9,6)  NOT NULL;

ALTER TABLE devices
    MODIFY device_type      ENUM('mobile', 'desktop', 'tablet') NOT NULL,
    MODIFY fingerprint_hash VARCHAR(64) NOT NULL;

ALTER TABLE rule_config
    MODIFY rule_name       VARCHAR(50)   NOT NULL,
    MODIFY enabled         BOOLEAN       NOT NULL,
    MODIFY threshold_value DECIMAL(12,4) NOT NULL;

ALTER TABLE users
    MODIFY full_name VARCHAR(150) NOT NULL,
    MODIFY email     VARCHAR(255) NOT NULL;

ALTER TABLE accounts
    MODIFY user_id      INT NOT NULL,
    MODIFY account_type ENUM('savings', 'current') NOT NULL,
    MODIFY balance      DECIMAL(14,2) NOT NULL,
    MODIFY status       ENUM('active', 'frozen', 'closed') NOT NULL;

ALTER TABLE merchants
    MODIFY merchant_name VARCHAR(200)  NOT NULL,
    MODIFY category_id   INT           NOT NULL,
    MODIFY city          VARCHAR(100)  NOT NULL,
    MODIFY latitude      DECIMAL(9,6)  NOT NULL,
    MODIFY longitude     DECIMAL(9,6)  NOT NULL;

ALTER TABLE transactions
    MODIFY account_id  INT NOT NULL,
    MODIFY merchant_id INT NOT NULL,
    MODIFY device_id   INT NOT NULL,
    MODIFY location_id INT NOT NULL,
    MODIFY amount      DECIMAL(12,2) NOT NULL,
    MODIFY txn_time    TIMESTAMP NOT NULL,
    MODIFY status      ENUM('success', 'pending', 'failed') NOT NULL;

ALTER TABLE alerts
    MODIFY txn_id     BIGINT       NOT NULL,
    MODIFY rule_name  VARCHAR(50)  NOT NULL,
    MODIFY risk_score DECIMAL(5,2) NOT NULL,
    MODIFY status     ENUM('open', 'investigated', 'confirmed', 'false_positive') NOT NULL;

ALTER TABLE behaviour_baselines MODIFY account_id INT NOT NULL;

ALTER TABLE login_history
    MODIFY account_id  INT NOT NULL,
    MODIFY device_id   INT NOT NULL,
    MODIFY location_id INT NOT NULL,
    MODIFY success     BOOLEAN NOT NULL;

ALTER TABLE audit_log
    MODIFY table_name VARCHAR(100) NOT NULL,
    MODIFY record_id  BIGINT       NOT NULL,
    MODIFY action     ENUM('INSERT', 'UPDATE', 'DELETE') NOT NULL;
```

```
-- ============================================================================
-- 3. UNIQUE   (candidate keys that are not the primary key)
-- ============================================================================
ALTER TABLE merchant_categories ADD UNIQUE (category_name);
ALTER TABLE locations           ADD CONSTRAINT uq_city_country UNIQUE (city, country);
ALTER TABLE devices             ADD UNIQUE (fingerprint_hash);
ALTER TABLE rule_config         ADD UNIQUE (rule_name);
ALTER TABLE users               ADD UNIQUE (email);
ALTER TABLE alerts              ADD UNIQUE (txn_id);          -- one alert per transaction
(1:1)
ALTER TABLE behaviour_baselines ADD UNIQUE (account_id);      -- one baseline per account (1:1)

-- ============================================================================
-- 4. DEFAULT
-- ============================================================================
-- Two syntaxes. ALTER COLUMN ... SET DEFAULT changes only the default and leaves the rest of
the column alone.
ALTER TABLE locations   ALTER COLUMN country      SET DEFAULT 'India';
ALTER TABLE rule_config ALTER COLUMN enabled      SET DEFAULT TRUE;
ALTER TABLE accounts
    ALTER COLUMN account_type SET DEFAULT 'savings',
    ALTER COLUMN balance      SET DEFAULT 0.00,
    ALTER COLUMN status       SET DEFAULT 'active';
ALTER TABLE merchants     ALTER COLUMN risk_score SET DEFAULT 0.00;
ALTER TABLE transactions  ALTER COLUMN status     SET DEFAULT 'success';
ALTER TABLE alerts        ALTER COLUMN status     SET DEFAULT 'open';
ALTER TABLE login_history ALTER COLUMN success    SET DEFAULT TRUE;

-- Timestamps: MODIFY restates the column so we can also add  ON UPDATE CURRENT_TIMESTAMP
(auto "last changed" stamp).
ALTER TABLE rule_config         MODIFY updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP;
ALTER TABLE users               MODIFY created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
ALTER TABLE accounts            MODIFY created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
ALTER TABLE account_devices
    MODIFY first_seen TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    MODIFY last_seen  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;
ALTER TABLE transactions        MODIFY txn_time   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP;
ALTER TABLE alerts              MODIFY created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
ALTER TABLE behaviour_baselines MODIFY updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP;
ALTER TABLE login_history       MODIFY login_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
ALTER TABLE audit_log           MODIFY changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;

-- ============================================================================
-- 5. CHECK   (enforced from MySQL 8.0.16 onward; older versions parse and silently ignore it)
-- ============================================================================
ALTER TABLE transactions ADD CONSTRAINT chk_txn_amount     CHECK (amount > 0);
ALTER TABLE accounts     ADD CONSTRAINT chk_accounts_balance CHECK (balance >= 0);
ALTER TABLE locations    ADD CONSTRAINT chk_loc_latitude   CHECK (latitude  BETWEEN  -90 AND
90);
ALTER TABLE locations    ADD CONSTRAINT chk_loc_longitude  CHECK (longitude BETWEEN -180 AND
180);

-- ============================================================================
-- 6. FOREIGN KEY   (referential integrity; the ON DELETE rule is a design decision, see
README)
-- ============================================================================
-- CASCADE  : delete the parent -> its children are deleted too (data that is meaningless
without the parent)
-- RESTRICT : refuse to delete a parent that still has children (data we must never silently
lose)
ALTER TABLE accounts
    ADD CONSTRAINT fk_accounts_user     FOREIGN KEY (user_id)     REFERENCES
users(user_id)                      ON DELETE CASCADE;
ALTER TABLE merchants
    ADD CONSTRAINT fk_merchants_category FOREIGN KEY (category_id) REFERENCES
```

```
merchant_categories(category_id)   ON DELETE RESTRICT;
ALTER TABLE account_devices
    ADD CONSTRAINT fk_ad_account        FOREIGN KEY (account_id)  REFERENCES
accounts(account_id)               ON DELETE CASCADE,
    ADD CONSTRAINT fk_ad_device         FOREIGN KEY (device_id)   REFERENCES
devices(device_id)                 ON DELETE CASCADE;
ALTER TABLE transactions
    ADD CONSTRAINT fk_txn_account       FOREIGN KEY (account_id)  REFERENCES
accounts(account_id)               ON DELETE CASCADE,
    ADD CONSTRAINT fk_txn_merchant      FOREIGN KEY (merchant_id) REFERENCES
merchants(merchant_id)             ON DELETE RESTRICT,
    ADD CONSTRAINT fk_txn_device        FOREIGN KEY (device_id)   REFERENCES
devices(device_id)                 ON DELETE RESTRICT,
    ADD CONSTRAINT fk_txn_location      FOREIGN KEY (location_id) REFERENCES
locations(location_id)             ON DELETE RESTRICT;
ALTER TABLE alerts
    ADD CONSTRAINT fk_alert_txn         FOREIGN KEY (txn_id)      REFERENCES
transactions(txn_id)               ON DELETE CASCADE;
ALTER TABLE behaviour_baselines
    ADD CONSTRAINT fk_bb_account        FOREIGN KEY (account_id)  REFERENCES
accounts(account_id)               ON DELETE CASCADE;
ALTER TABLE login_history
    ADD CONSTRAINT fk_lh_account        FOREIGN KEY (account_id)  REFERENCES
accounts(account_id)               ON DELETE CASCADE,
    ADD CONSTRAINT fk_lh_device         FOREIGN KEY (device_id)   REFERENCES
devices(device_id)                 ON DELETE RESTRICT,
    ADD CONSTRAINT fk_lh_location       FOREIGN KEY (location_id) REFERENCES
locations(location_id)             ON DELETE RESTRICT;
```

A.2 Detection views (week 09)

`week-09-views-indexes-dcl/sql/10_rule_views.sql`

```
-- ============================================================================
-- Week 9 — VIEWS: the six fraud rules, saved as named, reusable queries
-- 10_rule_views.sql
-- ============================================================================
-- A view is a stored SELECT. It holds no data of its own; every time you query it, MySQL runs
the SELECT.
-- Why use them here:
--   * each rule gets a NAME ("SELECT * FROM rule_new_device") instead of 20 lines pasted
around
--   * the logic lives in ONE place, so fixing a rule fixes every report that uses it
--   * Week 9's roles can be allowed to read a view without being allowed to read the tables
underneath
-- Every rule view returns one row per FLAGGED transaction, and reads its threshold from
rule_config
-- (and respects rule_config.enabled), so tuning is an UPDATE.
--
-- Syntax used beyond the lab list: WITH (common table expression = a named subquery you can
reuse inside one
-- statement) and LAG() (a window function = look at the previous row of the same group). Both
are explained in README.md.

USE sentinel;

-- ----------------------------------------------------------------------------
-- Rule 1 — high amount (z-score).   Week 5 by hand, Week 8 for all accounts, now a view.
-- ----------------------------------------------------------------------------
CREATE OR REPLACE VIEW rule_zscore AS
WITH account_stats AS (
    SELECT account_id,
           AVG(amount)         AS avg_amt,
           STDDEV_SAMP(amount) AS sd_amt
    FROM transactions
    GROUP BY account_id
```

```
    HAVING COUNT(*) >= 5
)
SELECT t.txn_id,
       t.account_id,
       t.amount,
       ROUND(s.avg_amt, 2) AS avg_amt,
       ROUND(s.sd_amt, 2)  AS stddev_amt,
       ROUND((t.amount - s.avg_amt) / s.sd_amt, 2) AS z_score,
       rc.threshold_value  AS threshold
FROM transactions t
JOIN account_stats s ON s.account_id = t.account_id
JOIN rule_config  rc ON rc.rule_name = 'zscore'
WHERE rc.enabled
  AND (t.amount - s.avg_amt) / NULLIF(s.sd_amt, 0) > rc.threshold_value;

-- ------------------------------------------------------------------------
-- Rule 2 — off-hours.   Flag a transaction whose hour of day holds fewer than threshold (2 %)
of the account's history.
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW rule_off_hours AS
WITH hour_counts AS (
    SELECT account_id, HOUR(txn_time) AS hr, COUNT(*) AS hour_txns
    FROM transactions
    GROUP BY account_id, HOUR(txn_time)
),
totals AS (
    SELECT account_id, COUNT(*) AS total_txns
    FROM transactions
    GROUP BY account_id
)
SELECT t.txn_id,
       t.account_id,
       HOUR(t.txn_time) AS txn_hour,
       ROUND(100 * h.hour_txns / tt.total_txns, 2) AS hour_pct,
       rc.threshold_value * 100                    AS threshold_pct
FROM transactions t
JOIN hour_counts h  ON h.account_id = t.account_id AND h.hr = HOUR(t.txn_time)
JOIN totals      tt ON tt.account_id = t.account_id
JOIN rule_config rc ON rc.rule_name = 'off_hours'
WHERE rc.enabled
  AND h.hour_txns / tt.total_txns < rc.threshold_value;

-- ------------------------------------------------------------------------
-- Rule 3 — new device.   The Week 7 anti-join (LEFT JOIN ... IS NULL).
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW rule_new_device AS
SELECT t.txn_id,
       t.account_id,
       t.device_id,
       d.fingerprint_hash,
       d.device_type,
       d.os
FROM transactions t
JOIN devices d                ON d.device_id = t.device_id
JOIN rule_config rc           ON rc.rule_name = 'new_device'
LEFT JOIN account_devices ad  ON ad.account_id = t.account_id AND ad.device_id = t.device_id
WHERE rc.enabled
  AND ad.account_id IS NULL;

-- ------------------------------------------------------------------------
-- Rule 4 — velocity.   The Week 7 self-join + Week 8 GROUP BY / HAVING.
-- Window length (minutes) and the limit both come from rule_config.
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW rule_velocity AS
SELECT t1.txn_id,
       t1.account_id,
       t1.txn_time,
       COUNT(*)                                AS txns_in_window,
```

```
       CAST(lim.threshold_value AS UNSIGNED)     AS velocity_limit,
       CAST(win.threshold_value AS UNSIGNED)     AS window_minutes
FROM transactions t1
JOIN rule_config lim ON lim.rule_name = 'velocity'
JOIN rule_config win ON win.rule_name = 'velocity_window'
JOIN transactions t2 ON t2.account_id = t1.account_id
                    AND t2.txn_time BETWEEN DATE_SUB(t1.txn_time, INTERVAL
CAST(win.threshold_value AS UNSIGNED) MINUTE)
                                        AND t1.txn_time
WHERE lim.enabled
GROUP BY t1.txn_id, t1.account_id, t1.txn_time, lim.threshold_value, win.threshold_value
HAVING COUNT(*) > velocity_limit;      -- HAVING may use the alias defined in the SELECT list

-- ------------------------------------------------------------------------
-- Rule 5 — impossible travel.   LAG() fetches the account's PREVIOUS transaction (location and
time) on the same row.
-- The distance is the Haversine formula written inline (Week 5). Week 10 replaces this view so
that it calls a stored
-- function, haversine_km(), instead of repeating the formula.
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW rule_impossible_travel AS
WITH with_previous AS (
    SELECT t.txn_id,
           t.account_id,
           t.txn_time,
           l.latitude  AS lat,
           l.longitude AS lon,
           LAG(l.latitude)  OVER w AS prev_lat,
           LAG(l.longitude) OVER w AS prev_lon,
           LAG(t.txn_time)  OVER w AS prev_time
    FROM transactions t
    JOIN locations l ON l.location_id = t.location_id
    WINDOW w AS (PARTITION BY t.account_id ORDER BY t.txn_time, t.txn_id)
),
with_distance AS (
    SELECT txn_id, account_id,
           2 * 6371 * ASIN(SQRT(POWER(SIN(RADIANS(lat - prev_lat) / 2), 2)
               + COS(RADIANS(prev_lat)) * COS(RADIANS(lat)) * POWER(SIN(RADIANS(lon -
prev_lon) / 2), 2))) AS distance_km,
           TIMESTAMPDIFF(SECOND, prev_time, txn_time) / 3600.0 AS hours_apart
    FROM with_previous
    WHERE prev_time IS NOT NULL
)
SELECT wd.txn_id,
       wd.account_id,
       ROUND(wd.distance_km, 2)                    AS distance_km,
       ROUND(wd.hours_apart * 60, 1)               AS minutes_diff,
       ROUND(wd.distance_km / wd.hours_apart, 0)   AS implied_speed_kmh,
       rc.threshold_value                          AS max_speed_kmh
FROM with_distance wd
JOIN rule_config rc ON rc.rule_name = 'impossible_travel'
WHERE rc.enabled
  AND wd.hours_apart > 0
  AND wd.distance_km > 50
  AND wd.distance_km / wd.hours_apart > rc.threshold_value;

-- ------------------------------------------------------------------------
-- Rule 6 — merchant risk.   Week 8's GROUP BY / HAVING / subquery, now using the REAL alerts
table.
-- Until alerts exist (Week 10) this view is empty, and that is correct: no alerts means no
merchant stands out.
-- A merchant must also have at least `merchant_risk_min_alerts` (3) alerts: with a tiny
platform average, ONE alert at a
-- merchant would otherwise make its alert rate several times the average and condemn all of
its transactions.
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW rule_merchant_risk AS
WITH merchant_stats AS (
```

```
    SELECT t.merchant_id, COUNT(*) AS total_txns, COUNT(a.alert_id) AS alert_count
    FROM transactions t
    LEFT JOIN alerts a ON a.txn_id = t.txn_id
    GROUP BY t.merchant_id
    HAVING COUNT(*) >= 3
),
platform AS (
    SELECT COUNT(a.alert_id) / COUNT(*) AS platform_rate
    FROM transactions t
    LEFT JOIN alerts a ON a.txn_id = t.txn_id
)
SELECT t.txn_id,
       t.account_id,
       m.merchant_id,
       m.merchant_name,
       ms.total_txns,
       ms.alert_count,
       ROUND(100 * ms.alert_count / ms.total_txns, 2)               AS merchant_alert_pct,
       ROUND(100 * p.platform_rate, 2)                              AS platform_alert_pct,
       ROUND((ms.alert_count / ms.total_txns) / p.platform_rate, 2)  AS risk_multiplier,
       rc.threshold_value                                           AS threshold
FROM transactions t
JOIN merchant_stats ms ON ms.merchant_id = t.merchant_id
JOIN merchants      m  ON m.merchant_id  = t.merchant_id
CROSS JOIN platform p
JOIN rule_config    rc ON rc.rule_name = 'merchant_risk'
JOIN rule_config    rm ON rm.rule_name = 'merchant_risk_min_alerts'
WHERE rc.enabled
  AND ms.alert_count >= rm.threshold_value          -- one stray alert must not condemn a
merchant
  AND (ms.alert_count / ms.total_txns) / NULLIF(p.platform_rate, 0) > rc.threshold_value;

-- ------------------------------------------------------------------------
-- Combined view — one row per suspicious transaction: how many rules fired, and why in plain
English.
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW v_transaction_risk AS
SELECT t.txn_id,
       t.account_id,
       t.amount,
       t.txn_time,
       (z.txn_id IS NOT NULL) + (oh.txn_id IS NOT NULL) + (nd.txn_id IS NOT NULL)
     + (v.txn_id IS NOT NULL) + (it.txn_id IS NOT NULL) + (mr.txn_id IS NOT NULL) AS
risk_score,
       CONCAT_WS('; ',
           IF(z.txn_id  IS NOT NULL, CONCAT('High amount (z=', z.z_score, ')'), NULL),
           IF(oh.txn_id IS NOT NULL, CONCAT('Off-hours (', oh.txn_hour, ':00, only ',
oh.hour_pct, '% of history)'), NULL),
           IF(nd.txn_id IS NOT NULL, CONCAT('New device (', LEFT(nd.fingerprint_hash, 8),
'...)'), NULL),
           IF(v.txn_id  IS NOT NULL, CONCAT('Velocity spike (', v.txns_in_window, ' in ',
v.window_minutes, ' min)'), NULL),
           IF(it.txn_id IS NOT NULL, CONCAT('Impossible travel (', it.distance_km, ' km in ',
it.minutes_diff, ' min)'), NULL),
           IF(mr.txn_id IS NOT NULL, CONCAT('Merchant risk (', mr.risk_multiplier, 'x platform
average)'), NULL)
       ) AS reasons
FROM transactions t
LEFT JOIN rule_zscore            z  ON z.txn_id  = t.txn_id
LEFT JOIN rule_off_hours         oh ON oh.txn_id = t.txn_id
LEFT JOIN rule_new_device        nd ON nd.txn_id = t.txn_id
LEFT JOIN rule_velocity          v  ON v.txn_id  = t.txn_id
LEFT JOIN rule_impossible_travel it ON it.txn_id = t.txn_id
LEFT JOIN rule_merchant_risk     mr ON mr.txn_id = t.txn_id
WHERE z.txn_id IS NOT NULL OR oh.txn_id IS NOT NULL OR nd.txn_id IS NOT NULL
   OR v.txn_id IS NOT NULL OR it.txn_id IS NOT NULL OR mr.txn_id IS NOT NULL;
```

A.3 Reporting views (week 09)

`week-09-views-indexes-dcl/sql/20_reporting_views.sql`

```
-- ============================================================================
-- Week 9 — VIEWS for reports: what an analyst or dashboard reads
-- 20_reporting_views.sql
-- ============================================================================
-- Joins several tables so a reader never needs to know the table layout (this is "logical data
independence").
-- Because MySQL views run with their DEFINER's rights, a role can be given SELECT on a view
without any right
-- to the tables underneath. 40_roles_dcl.sql relies on that.

USE sentinel;

-- Open alerts with full context. Empty until Week 10 creates alerts.
CREATE OR REPLACE VIEW v_active_alerts AS
SELECT al.alert_id,
       al.txn_id,
       al.rule_name,
       al.risk_score,
       al.reasons,
       al.status,
       al.created_at,
       t.amount,
       t.txn_time,
       a.account_id,
       u.full_name         AS user_name,
       u.email,
       m.merchant_name,
       mc.category_name    AS merchant_category,
       l.city              AS txn_city,
       d.device_type,
       d.os                AS device_os
FROM alerts al
JOIN transactions        t  ON t.txn_id        = al.txn_id
JOIN accounts            a  ON a.account_id    = t.account_id
JOIN users               u  ON u.user_id       = a.user_id
JOIN merchants           m  ON m.merchant_id   = t.merchant_id
JOIN merchant_categories mc ON mc.category_id  = m.category_id
JOIN locations           l  ON l.location_id   = t.location_id
JOIN devices             d  ON d.device_id     = t.device_id
WHERE al.status = 'open';

-- Accounts with the most alerts
CREATE OR REPLACE VIEW v_top_risky_users AS
SELECT a.account_id,
       u.full_name,
       u.email,
       COUNT(al.alert_id)            AS total_alerts,
       ROUND(AVG(al.risk_score), 2)  AS avg_risk_score,
       MAX(al.risk_score)            AS max_risk_score
FROM alerts al
JOIN transactions t ON t.txn_id     = al.txn_id
JOIN accounts     a ON a.account_id = t.account_id
JOIN users        u ON u.user_id    = a.user_id
GROUP BY a.account_id, u.full_name, u.email
ORDER BY total_alerts DESC, a.account_id
LIMIT 50;

-- Alerts per city per day
CREATE OR REPLACE VIEW v_alerts_by_city_day AS
SELECT l.city,
       DATE(t.txn_time)            AS txn_date,
       COUNT(*)                    AS alert_count,
```

```
       ROUND(AVG(al.risk_score), 2) AS avg_risk_score
FROM alerts al
JOIN transactions t ON t.txn_id      = al.txn_id
JOIN locations    l ON l.location_id = t.location_id
GROUP BY l.city, DATE(t.txn_time)
ORDER BY txn_date DESC, alert_count DESC;

-- Daily activity summary. Works from Week 4 on, because it needs no alerts.
CREATE OR REPLACE VIEW v_daily_txn_summary AS
SELECT DATE(txn_time)              AS txn_date,
       COUNT(*)                    AS total_txns,
       ROUND(SUM(amount), 2)       AS total_amount,
       ROUND(AVG(amount), 2)       AS avg_amount,
       MIN(amount)                 AS min_amount,
       MAX(amount)                 AS max_amount,
       COUNT(DISTINCT account_id)  AS unique_accounts
FROM transactions
GROUP BY DATE(txn_time)
ORDER BY txn_date DESC;
```

A.4 Indexes (week 09)

`week-09-views-indexes-dcl/sql/30_indexes.sql`

```
-- ============================================================================
-- Week 9 — INDEXES
-- 30_indexes.sql — speed up the access paths the rules actually use
-- ============================================================================
-- An index is a sorted copy of one or more columns (a B+ tree) that lets MySQL jump to
matching rows instead of
-- reading the whole table. Cost: extra disk space, and every INSERT/UPDATE must maintain the
index too.
-- Rule of thumb: index the columns you filter, join or sort by, in the order you use them.
--
-- Measure before and after: demo/explain_before_after.sql, results in docs/
benchmark_measured.md.
--
-- Deliberately NOT created: an index on alerts(txn_id) or behaviour_baselines(account_id).
Both columns are already
-- UNIQUE, and a UNIQUE constraint IS an index. A second one would only slow down writes.

USE sentinel;

-- transactions: almost every query filters by account (and then time) or by merchant
CREATE INDEX idx_txn_account_time  ON transactions (account_id, txn_time);   -- velocity,
travel, baselines, per-account history
CREATE INDEX idx_txn_merchant_time ON transactions (merchant_id, txn_time);  -- merchant risk
CREATE INDEX idx_txn_location      ON transactions (location_id);
CREATE INDEX idx_txn_time          ON transactions (txn_time);               -- date-range
reports

-- alerts: dashboards filter by status then sort by newest
CREATE INDEX idx_alert_status ON alerts (status, created_at);
CREATE INDEX idx_alert_rule   ON alerts (rule_name);

-- login_history and audit_log: lookups by account / table, newest first
CREATE INDEX idx_lh_account_time   ON login_history (account_id, login_time);
CREATE INDEX idx_audit_table_time  ON audit_log (table_name, changed_at);

-- account_devices: "which accounts use this device?" (the primary key covers the other
direction)
CREATE INDEX idx_ad_device ON account_devices (device_id);
```

A.5 Roles and grants (week 09)

`week-09-views-indexes-dcl/sql/40_roles_dcl.sql`

```
-- ============================================================================
-- Week 9 — DCL: roles, GRANT, REVOKE   (principle of least privilege)
-- 40_roles_dcl.sql
-- ============================================================================
-- DCL = Data Control Language: who may do what. A ROLE is a named bundle of privileges; you
GRANT the role to users.
--
--   fraud_analyst : read the report/rule views, and close alerts. NO access to the raw tables.
--   txn_app       : the payment application - may insert and read transactions, nothing else.
--   api_reader    : the read-only dashboard/API login - the report views and the rule
thresholds, nothing else.
--   db_admin      : everything inside `sentinel`.
--
-- The passwords below are for a local teaching database only. Never commit real credentials.

USE sentinel;

CREATE ROLE IF NOT EXISTS 'fraud_analyst';
CREATE ROLE IF NOT EXISTS 'txn_app';
CREATE ROLE IF NOT EXISTS 'api_reader';
CREATE ROLE IF NOT EXISTS 'db_admin';

-- fraud_analyst: SELECT on views only ...
GRANT SELECT ON sentinel.v_active_alerts      TO 'fraud_analyst';
GRANT SELECT ON sentinel.v_top_risky_users    TO 'fraud_analyst';
GRANT SELECT ON sentinel.v_alerts_by_city_day TO 'fraud_analyst';
GRANT SELECT ON sentinel.v_daily_txn_summary  TO 'fraud_analyst';
GRANT SELECT ON sentinel.v_transaction_risk   TO 'fraud_analyst';
GRANT SELECT ON sentinel.rule_zscore            TO 'fraud_analyst';
GRANT SELECT ON sentinel.rule_off_hours         TO 'fraud_analyst';
GRANT SELECT ON sentinel.rule_new_device        TO 'fraud_analyst';
GRANT SELECT ON sentinel.rule_velocity          TO 'fraud_analyst';
GRANT SELECT ON sentinel.rule_impossible_travel TO 'fraud_analyst';
GRANT SELECT ON sentinel.rule_merchant_risk     TO 'fraud_analyst';
-- ... plus the right to CLOSE an alert. A column-level UPDATE grant is not enough on its own:
MySQL also needs to be
-- allowed to READ the columns in the WHERE clause, so those get a column-level SELECT.
GRANT SELECT (alert_id, txn_id, status, reviewed_at) ON sentinel.alerts TO 'fraud_analyst';
GRANT UPDATE (status, reviewed_at)                   ON sentinel.alerts TO 'fraud_analyst';

-- txn_app: the application inserts transactions and can read them back
GRANT INSERT, SELECT ON sentinel.transactions TO 'txn_app';

-- api_reader: read-only; the four report views plus the thresholds. No raw tables, no writes.
GRANT SELECT ON sentinel.v_active_alerts      TO 'api_reader';
GRANT SELECT ON sentinel.v_daily_txn_summary  TO 'api_reader';
GRANT SELECT ON sentinel.v_top_risky_users    TO 'api_reader';
GRANT SELECT ON sentinel.v_alerts_by_city_day TO 'api_reader';
GRANT SELECT ON sentinel.rule_config          TO 'api_reader';

-- db_admin: everything in this database (but not other databases)
GRANT ALL PRIVILEGES ON sentinel.* TO 'db_admin';

-- Users, and which role each gets
CREATE USER IF NOT EXISTS 'analyst_user'@'%' IDENTIFIED BY 'analyst_2026';
CREATE USER IF NOT EXISTS 'app_user'@'%'     IDENTIFIED BY 'app_2026';
CREATE USER IF NOT EXISTS 'admin_user'@'%'   IDENTIFIED BY 'admin_2026';
CREATE USER IF NOT EXISTS 'api_user'@'%'     IDENTIFIED BY 'api_2026';

GRANT 'fraud_analyst' TO 'analyst_user'@'%';
GRANT 'txn_app'       TO 'app_user'@'%';
GRANT 'db_admin'      TO 'admin_user'@'%';
```

```
GRANT 'api_reader'    TO 'api_user'@'%';

-- A role only takes effect at login if it is a DEFAULT role
SET DEFAULT ROLE 'fraud_analyst' TO 'analyst_user'@'%';
SET DEFAULT ROLE 'txn_app'       TO 'app_user'@'%';
SET DEFAULT ROLE 'db_admin'      TO 'admin_user'@'%';
SET DEFAULT ROLE 'api_reader'    TO 'api_user'@'%';
```

A.6 Scoring procedure `score_transaction` (week 10)

`week-10-procedures-triggers/sql/20_score_transaction.sql`

```
-- ============================================================================
-- Week 10 — STORED PROCEDURE: score one transaction against all six rules
-- 20_score_transaction.sql
-- ============================================================================
-- CALL score_transaction(12052);   -- scores that transaction and, if any rule fired, writes/
updates its alert.
--
-- This is the SAME logic as the six rule views from Week 9, written procedurally for ONE
transaction (a view
-- scans every transaction, which is far too slow to run on each insert). Each rule block
follows one pattern:
--     read the rule's switch + threshold from rule_config  ->  compute the evidence  ->
compare  ->  add a point and a reason.
-- Only local variables are used (DECLARE). Session variables (@x) would leak between calls.
-- The alert's `reasons` column is a JSON array of readable strings; `risk_score` is the number
of rules that fired.

USE sentinel;

DROP PROCEDURE IF EXISTS score_transaction;
DELIMITER //
CREATE PROCEDURE score_transaction(IN p_txn_id BIGINT)
BEGIN
    -- the transaction being scored
    DECLARE v_account_id  INT;
    DECLARE v_amount      DECIMAL(12,2);
    DECLARE v_txn_time    TIMESTAMP;
    DECLARE v_merchant_id INT;
    DECLARE v_device_id   INT;
    DECLARE v_location_id INT;
    -- the verdict
    DECLARE v_score   INT  DEFAULT 0;
    DECLARE v_reasons JSON DEFAULT JSON_ARRAY();
    -- scratch space
    DECLARE v_enabled   BOOLEAN;
    DECLARE v_threshold DECIMAL(12,4);
    DECLARE v_window    INT;
    DECLARE v_n         INT;
    DECLARE v_avg       DOUBLE;
    DECLARE v_sd        DOUBLE;
    DECLARE v_value     DOUBLE;
    DECLARE v_fp        VARCHAR(8);
    DECLARE v_prev_lat  DECIMAL(9,6);
    DECLARE v_prev_lon  DECIMAL(9,6);
    DECLARE v_prev_time TIMESTAMP;
    DECLARE v_lat       DECIMAL(9,6);
    DECLARE v_lon       DECIMAL(9,6);
    DECLARE v_km        DOUBLE;
    DECLARE v_hours     DOUBLE;
    DECLARE v_min_alerts DECIMAL(12,4);
    DECLARE v_m_total   INT;
    DECLARE v_m_alerts  INT;
    DECLARE v_p_total   INT;
    DECLARE v_p_alerts  INT;
```

```
    -- Load the transaction. If there is no such row the variables stay NULL.
    SELECT account_id, amount, txn_time, merchant_id, device_id, location_id
      INTO v_account_id, v_amount, v_txn_time, v_merchant_id, v_device_id, v_location_id
      FROM transactions WHERE txn_id = p_txn_id;
    IF v_account_id IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'score_transaction: no such transaction';
    END IF;

    -- ---------------------------------------------------------------- Rule 1: high amount (z-
score)
    SELECT enabled, threshold_value INTO v_enabled, v_threshold FROM rule_config WHERE
rule_name = 'zscore';
    IF v_enabled THEN
        SELECT AVG(amount), STDDEV_SAMP(amount), COUNT(*) INTO v_avg, v_sd, v_n
          FROM transactions WHERE account_id = v_account_id;
        IF v_n >= 5 AND v_sd > 0 THEN
            SET v_value = (v_amount - v_avg) / v_sd;
            IF v_value > v_threshold THEN
                SET v_score = v_score + 1;
                SET v_reasons = JSON_ARRAY_APPEND(v_reasons, '$', CONCAT('High amount (z=',
ROUND(v_value, 2), ')'));
            END IF;
        END IF;
    END IF;

    -- ---------------------------------------------------------------- Rule 2: off-hours
    SELECT enabled, threshold_value INTO v_enabled, v_threshold FROM rule_config WHERE
rule_name = 'off_hours';
    IF v_enabled THEN
        SELECT SUM(HOUR(txn_time) = HOUR(v_txn_time)), COUNT(*) INTO v_value, v_n
          FROM transactions WHERE account_id = v_account_id;
        IF v_value / v_n < v_threshold THEN
            SET v_score = v_score + 1;
            SET v_reasons = JSON_ARRAY_APPEND(v_reasons, '$',
                CONCAT('Off-hours (', HOUR(v_txn_time), ':00, only ', ROUND(100 * v_value /
v_n, 2), '% of history)'));
        END IF;
    END IF;

    -- ---------------------------------------------------------------- Rule 3: new device
    SELECT enabled INTO v_enabled FROM rule_config WHERE rule_name = 'new_device';
    IF v_enabled THEN
        IF NOT EXISTS (SELECT 1 FROM account_devices WHERE account_id = v_account_id AND
device_id = v_device_id) THEN
            SELECT LEFT(fingerprint_hash, 8) INTO v_fp FROM devices WHERE device_id =
v_device_id;
            SET v_score = v_score + 1;
            SET v_reasons = JSON_ARRAY_APPEND(v_reasons, '$', CONCAT('New device (', v_fp,
'...)'));
        END IF;
    END IF;

    -- ---------------------------------------------------------------- Rule 4: velocity
    SELECT enabled, threshold_value INTO v_enabled, v_threshold FROM rule_config WHERE
rule_name = 'velocity';
    IF v_enabled THEN
        SELECT CAST(threshold_value AS UNSIGNED) INTO v_window FROM rule_config WHERE rule_name
= 'velocity_window';
        SELECT COUNT(*) INTO v_n
          FROM transactions
         WHERE account_id = v_account_id
           AND txn_time BETWEEN DATE_SUB(v_txn_time, INTERVAL v_window MINUTE) AND v_txn_time;
        IF v_n > v_threshold THEN
            SET v_score = v_score + 1;
            SET v_reasons = JSON_ARRAY_APPEND(v_reasons, '$', CONCAT('Velocity spike (', v_n, '
in ', v_window, ' min)'));
        END IF;
```

```
    END IF;

    -- ---------------------------------------------------------- Rule 5: impossible
travel
    SELECT enabled, threshold_value INTO v_enabled, v_threshold FROM rule_config WHERE
rule_name = 'impossible_travel';
    IF v_enabled THEN
        -- the account's previous transaction (same ordering the view's LAG() uses)
        SELECT l.latitude, l.longitude, t.txn_time INTO v_prev_lat, v_prev_lon, v_prev_time
          FROM transactions t JOIN locations l ON l.location_id = t.location_id
         WHERE t.account_id = v_account_id
           AND (t.txn_time < v_txn_time OR (t.txn_time = v_txn_time AND t.txn_id < p_txn_id))
         ORDER BY t.txn_time DESC, t.txn_id DESC
         LIMIT 1;
        IF v_prev_time IS NOT NULL THEN
            SELECT latitude, longitude INTO v_lat, v_lon FROM locations WHERE location_id =
v_location_id;
            SET v_km    = haversine_km(v_prev_lat, v_prev_lon, v_lat, v_lon);
            SET v_hours = TIMESTAMPDIFF(SECOND, v_prev_time, v_txn_time) / 3600.0;
            IF v_hours > 0 AND v_km > 50 AND v_km / v_hours > v_threshold THEN
                SET v_score = v_score + 1;
                SET v_reasons = JSON_ARRAY_APPEND(v_reasons, '$',
                    CONCAT('Impossible travel (', ROUND(v_km, 2), ' km in ', ROUND(v_hours *
60, 1), ' min)'));
            END IF;
        END IF;
    END IF;

    -- ---------------------------------------------------------- Rule 6: merchant risk
    SELECT enabled, threshold_value INTO v_enabled, v_threshold FROM rule_config WHERE
rule_name = 'merchant_risk';
    IF v_enabled THEN
        SELECT COUNT(*) INTO v_m_total  FROM transactions WHERE merchant_id = v_merchant_id;
        SELECT COUNT(*) INTO v_m_alerts FROM alerts al JOIN transactions t ON t.txn_id =
al.txn_id WHERE t.merchant_id = v_merchant_id;
        SELECT COUNT(*) INTO v_p_total  FROM transactions;
        SELECT COUNT(*) INTO v_p_alerts FROM alerts;
        SELECT threshold_value INTO v_min_alerts FROM rule_config WHERE rule_name =
'merchant_risk_min_alerts';
        IF v_m_total >= 3 AND v_m_alerts >= v_min_alerts AND v_p_alerts > 0 THEN
            SET v_value = (v_m_alerts / v_m_total) / (v_p_alerts / v_p_total);
            IF v_value > v_threshold THEN
                SET v_score = v_score + 1;
                SET v_reasons = JSON_ARRAY_APPEND(v_reasons, '$', CONCAT('Merchant risk (',
ROUND(v_value, 2), 'x platform average)'));
            END IF;
        END IF;
    END IF;

    -- ---------------------------------------------------------- Verdict
    IF v_score > 0 THEN
        INSERT INTO alerts (txn_id, rule_name, risk_score, reasons, status)
        VALUES (p_txn_id, 'combined', v_score, v_reasons, 'open')
        ON DUPLICATE KEY UPDATE risk_score = v_score, reasons = v_reasons;   -- re-scoring
updates the existing alert
    ELSE
        DELETE FROM alerts WHERE txn_id = p_txn_id AND status = 'open';      -- no longer
suspicious (e.g. a threshold was loosened)
    END IF;
END //
DELIMITER ;

-- Score many transactions in one go (used to back-fill history that was loaded before the
trigger existed).
DROP PROCEDURE IF EXISTS backfill_alerts;
DELIMITER //
CREATE PROCEDURE backfill_alerts(IN p_from_txn BIGINT, IN p_to_txn BIGINT)
BEGIN
```

```
    DECLARE v_id BIGINT DEFAULT p_from_txn;
    WHILE v_id <= p_to_txn DO
        IF EXISTS (SELECT 1 FROM transactions WHERE txn_id = v_id) THEN
            CALL score_transaction(v_id);
        END IF;
        SET v_id = v_id + 1;
    END WHILE;
END //
DELIMITER ;
```

A.7 Triggers (week 10)

`week-10-procedures-triggers/sql/30_triggers.sql`

```
-- ============================================================================
-- Week 10 — TRIGGERS
-- 30_triggers.sql — code the database runs by itself when rows change
-- ============================================================================
-- A trigger is attached to a table + an event (INSERT / UPDATE / DELETE) + a moment (BEFORE /
AFTER).
-- Inside it, NEW is the row being written and OLD is the row as it was (UPDATE/DELETE only).
-- Nobody calls a trigger. That is the whole point: no application can forget to run the fraud
check.

USE sentinel;

-- 1. AFTER INSERT on transactions: score every new transaction the moment it is stored.
--    It is AFTER (not BEFORE) because the scoring procedure reads the transaction, and a
BEFORE trigger runs before
--    the row exists. A row-level trigger fires once per inserted row, even for a multi-row
INSERT.
DROP TRIGGER IF EXISTS trg_after_txn_insert;
DELIMITER //
CREATE TRIGGER trg_after_txn_insert
AFTER INSERT ON transactions
FOR EACH ROW
BEGIN
    CALL score_transaction(NEW.txn_id);
END //
DELIMITER ;

-- 2. BEFORE UPDATE on alerts: when an analyst moves an alert out of 'open', stamp the review
time automatically.
--    BEFORE lets us change NEW before it is written (SET NEW.col = ...). An AFTER trigger
could not.
DROP TRIGGER IF EXISTS trg_before_alert_update;
DELIMITER //
CREATE TRIGGER trg_before_alert_update
BEFORE UPDATE ON alerts
FOR EACH ROW
BEGIN
    IF OLD.status = 'open' AND NEW.status <> 'open' AND NEW.reviewed_at IS NULL THEN
        SET NEW.reviewed_at = CURRENT_TIMESTAMP;
    END IF;
END //
DELIMITER ;

-- 3. AFTER UPDATE on alerts: write every status change to audit_log (who, when, from what, to
what).
--    Triggers run with the DEFINER's rights, so an analyst who can only UPDATE alerts still
causes an audit_log row.
--    changed_by uses USER(), not CURRENT_USER(): inside a trigger CURRENT_USER() is the
definer (root), which would
--    make every audit row blame root. (The original project had exactly this bug.)
DROP TRIGGER IF EXISTS trg_after_alert_update;
DELIMITER //
```

```
CREATE TRIGGER trg_after_alert_update
AFTER UPDATE ON alerts
FOR EACH ROW
BEGIN
    IF OLD.status <> NEW.status THEN
        INSERT INTO audit_log (table_name, record_id, action, old_value_json, new_value_json,
changed_by)
        VALUES ('alerts', NEW.alert_id, 'UPDATE',
                JSON_OBJECT('status', OLD.status),
                JSON_OBJECT('status', NEW.status),
                USER());
    END IF;
END //
DELIMITER ;
```

A.8 `transfer_funds` (week 10)

`week-10-procedures-triggers/sql/50_transfer_funds.sql`

```
-- ============================================================================
-- Week 10 — TRANSACTION MANAGEMENT: transfer_funds()
-- 50_transfer_funds.sql — move money between two accounts, all-or-nothing
-- ============================================================================
-- ACID in one procedure:
--   Atomicity   debit + credit + the transaction record happen together or not at all
(COMMIT / ROLLBACK)
--   Consistency balances never go negative (checked here AND by the CHECK constraint from Week
3)
--   Isolation   SELECT ... FOR UPDATE locks the two account rows, so two transfers cannot both
spend the same money
--   Durability  after COMMIT the change survives a crash (InnoDB redo log)
-- Locks are always taken in ASCENDING account_id order. If A->B and B->A ran at the same time
and each locked its
-- own account first, they would wait for each other forever (deadlock). One agreed order makes
that impossible.
--
-- The debit is also recorded in `transactions` (merchant 1 stands in for "transfer"), so the
AFTER INSERT trigger
-- from file 30 scores it like any other payment. Only the sender gets a row: money RECEIVED is
not a purchase and
-- must not distort the receiver's spending statistics.

USE sentinel;

DROP PROCEDURE IF EXISTS transfer_funds;
DELIMITER //
CREATE PROCEDURE transfer_funds(
    IN p_from_account INT,
    IN p_to_account   INT,
    IN p_amount       DECIMAL(12,2),
    IN p_device_id    INT,
    IN p_location_id  INT
)
BEGIN
    DECLARE v_from_balance DECIMAL(14,2);
    DECLARE v_from_status  VARCHAR(20);
    DECLARE v_to_status    VARCHAR(20);

    -- Any unexpected error (bad foreign key, deadlock, ...): undo everything, then re-raise it
to the caller.
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;
```

```
    -- Validate before touching anything.
    IF p_amount IS NULL OR p_amount <= 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transfer amount must be positive';
    END IF;
    IF p_from_account = p_to_account THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Cannot transfer to the same account';
    END IF;

    START TRANSACTION;

    -- Lock both rows, lowest id first.
    IF p_from_account < p_to_account THEN
        SELECT balance, status INTO v_from_balance, v_from_status FROM accounts WHERE
account_id = p_from_account FOR UPDATE;
        SELECT status          INTO v_to_status                   FROM accounts WHERE
account_id = p_to_account   FOR UPDATE;
    ELSE
        SELECT status          INTO v_to_status                   FROM accounts WHERE
account_id = p_to_account   FOR UPDATE;
        SELECT balance, status INTO v_from_balance, v_from_status FROM accounts WHERE
account_id = p_from_account FOR UPDATE;
    END IF;

    IF v_from_status IS NULL OR v_to_status IS NULL THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Source or destination account does not
exist';
    END IF;
    IF v_from_status <> 'active' THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Source account is not active';
    END IF;
    IF v_to_status <> 'active' THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Destination account is not active';
    END IF;
    IF v_from_balance < p_amount THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient funds';
    END IF;

    UPDATE accounts SET balance = balance - p_amount WHERE account_id = p_from_account;
    UPDATE accounts SET balance = balance + p_amount WHERE account_id = p_to_account;

    INSERT INTO transactions (account_id, merchant_id, device_id, location_id, amount, status)
    VALUES (p_from_account, 1, p_device_id, p_location_id, p_amount, 'success');

    COMMIT;

    SELECT 'Transfer completed' AS result, p_amount AS amount,
           (SELECT balance FROM accounts WHERE account_id = p_from_account) AS
sender_balance_now,
           (SELECT balance FROM accounts WHERE account_id = p_to_account)   AS
receiver_balance_now;
END //
DELIMITER ;
```

Appendix B. Captured Terminal Output

Each figure was produced by running the shown statement against the live `sentinel-db` container (MySQL 8.4) and capturing the unedited result on 21 Oct 2026. Figures B.11 and B.12 are the repository test runs.

```
mysql> SHOW FULL TABLES;

+------------------------+------------+
| Tables_in_sentinel     | Table_type |
+------------------------+------------+
| account_devices        | BASE TABLE |
| accounts               | BASE TABLE |
| alerts                 | BASE TABLE |
| audit_log              | BASE TABLE |
| behaviour_baselines    | BASE TABLE |
| devices                | BASE TABLE |
| locations              | BASE TABLE |
| login_history          | BASE TABLE |
| merchant_categories    | BASE TABLE |
| merchants              | BASE TABLE |
| rule_config            | BASE TABLE |
| rule_impossible_travel | VIEW       |
| rule_merchant_risk     | VIEW       |
| rule_new_device        | VIEW       |
| rule_off_hours         | VIEW       |
| rule_velocity          | VIEW       |
| rule_zscore            | VIEW       |
| transactions           | BASE TABLE |
| users                  | BASE TABLE |
| v_active_alerts        | VIEW       |
| v_alerts_by_city_day   | VIEW       |
| v_daily_txn_summary    | VIEW       |
| v_top_risky_users      | VIEW       |
| v_transaction_risk     | VIEW       |
+------------------------+------------+
```

Figure B.1 Thirteen base tables and eleven views.

```
mysql> DESCRIBE transactions;

+--------------+----------------------------------+------+-----+-------------------+-------------------+
| Field        | Type                             | Null | Key | Default           | Extra             |
+--------------+----------------------------------+------+-----+-------------------+-------------------+
| txn_id       | bigint                           | NO   | PRI | NULL              | auto_increment    |
| account_id   | int                              | NO   | MUL | NULL              |                   |
| merchant_id  | int                              | NO   | MUL | NULL              |                   |
| device_id    | int                              | NO   | MUL | NULL              |                   |
| location_id  | int                              | NO   | MUL | NULL              |                   |
| amount       | decimal(12,2)                    | NO   |     | NULL              |                   |
| txn_time     | timestamp                        | NO   | MUL | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| status       | enum('success','pending','failed') | NO   |     | success           |                   |
| metadata_json | json                             | YES  |     | NULL              |                   |
+--------------+----------------------------------+------+-----+-------------------+-------------------+
```

Figure B.2 Structure of `transactions`.

```
mysql> SELECT * FROM rule_config;

+---------+------------------------+---------+-----------------+-----------------------------------------------------------------------------------------------------------------------------------------------------------+---------------------+
| rule_id | rule_name              | enabled | threshold_value | parameters_json                                                                                                                                           | updated_at          |
+---------+------------------------+---------+-----------------+-----------------------------------------------------------------------------------------------------------------------------------------------------------+---------------------+
|       1 | zscore                 |       1 |          3.0000 | {"description": "Z-score above which an amount is anomalous"}                                                                                             | 2026-10-21 17:24:44 |
|       2 | off_hours              |       1 |          0.0200 | {"description": "Flag if the account has fewer than 2% of its transactions in this hour"}                                                                 | 2026-10-21 16:38:37 |
|       3 | new_device             |       1 |          1.0000 | {"description": "Flag if the device was never seen on this account"}                                                                                      | 2026-10-21 17:24:42 |
|       4 | velocity               |       1 |          5.0000 | {"description": "Max transactions allowed inside the velocity window"}                                                                                    | 2026-10-21 16:38:37 |
|       5 | velocity_window        |       1 |          5.0000 | {"description": "Velocity window length in minutes"}                                                                                                      | 2026-10-21 16:38:37 |
|       6 | impossible_travel      |       1 |        900.0000 | {"description": "Max plausible speed in km/h between two transactions"}                                                                                   | 2026-10-21 16:38:37 |
|       7 | merchant_risk          |       1 |          2.0000 | {"description": "Flag if a merchant alert rate is this many times the platform average"}                                                                  | 2026-10-21 16:38:37 |
|       8 | merchant_risk_min_alerts |       1 |          3.0000 | {"description": "A merchant needs at least this many alerts before it can be called risky (one alert proves nothing)"} | 2026-10-21 16:38:37 |
+---------+------------------------+---------+-----------------+-----------------------------------------------------------------------------------------------------------------------------------------------------------+---------------------+
```

Figure B.3 The eight `rule_config` rows.

```
mysql> INSERT INTO transactions (account_id,merchant_id,device_id,location_id,amount,status) VALUES (1,1,1,1,-50,'success');

ERROR 3819 (HY000) at line 1: Check constraint 'chk_txn_amount' is violated.
```

Figure B.4 A `CHECK` constraint rejecting a negative amount.

```
mysql> SHOW GRANTS FOR api_reader@'%';

+-----------------------------------------------------------------------------+
| Grants for api_reader@%                                                     |
+-----------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `api_reader`@`%`                                      |
| GRANT SELECT ON `sentinel`.`rule_config` TO `api_reader`@`%`                |
| GRANT SELECT ON `sentinel`.`v_active_alerts` TO `api_reader`@`%`            |
| GRANT SELECT ON `sentinel`.`v_alerts_by_city_day` TO `api_reader`@`%`       |
| GRANT SELECT ON `sentinel`.`v_daily_txn_summary` TO `api_reader`@`%`        |
| GRANT SELECT ON `sentinel`.`v_top_risky_users` TO `api_reader`@`%`          |
+-----------------------------------------------------------------------------+
```

Figure B.5 Least-privilege grants of `api_reader`.

```
mysql> SELECT TRIGGER_NAME,EVENT_MANIPULATION ev,EVENT_OBJECT_TABLE tbl,ACTION_TIMING timing FROM information_schema.TRIGGERS WHERE TRIGGER_SCHEMA='sentinel';

+-------------------------+--------+-------------+--------+
| TRIGGER_NAME            | ev     | tbl         | timing |
+-------------------------+--------+-------------+--------+
| trg_before_alert_update | UPDATE | alerts      | BEFORE |
| trg_after_alert_update  | UPDATE | alerts      | AFTER  |
| trg_after_txn_insert    | INSERT | transactions | AFTER  |
+-------------------------+--------+-------------+--------+
```

Figure B.6 Triggers registered in the schema.

```
mysql> SELECT ROUTINE_NAME,ROUTINE_TYPE FROM information_schema.ROUTINES WHERE ROUTINE_SCHEMA='sentinel';

+-----------------------------+--------------+
| ROUTINE_NAME                | ROUTINE_TYPE |
+-----------------------------+--------------+
| haversine_km                | FUNCTION     |
| backfill_alerts             | PROCEDURE    |
| refresh_behaviour_baselines | PROCEDURE    |
| score_transaction           | PROCEDURE    |
| transfer_funds              | PROCEDURE    |
+-----------------------------+--------------+
```

Figure B.7 Stored routines.

```
mysql> SELECT alert_id,txn_id,rule_name,risk_score,status,reasons FROM alerts WHERE txn_id BETWEEN 12041 AND 12067 ORDER BY alert_id LIMIT 6;

+----------+--------+-----------+------------+--------+-----------------------------------------------------+
| alert_id | txn_id | rule_name | risk_score | status | reasons                                             |
+----------+--------+-----------+------------+--------+-----------------------------------------------------+
|        1 |  12042 | combined  |       1.00 | open   | ["Impossible travel (1767.65 km in 10 min)"]        |
|        2 |  12043 | combined  |       1.00 | open   | ["Off-hours (2:00, only 1.92% of history)"]         |
|        3 |  12049 | combined  |       1.00 | open   | ["Velocity spike (6 in 5 min)"]                     |
|        4 |  12050 | combined  |       1.00 | open   | ["Velocity spike (7 in 5 min)"]                     |
|        5 |  12051 | combined  |       1.00 | open   | ["Velocity spike (8 in 5 min)"]                     |
|        6 |  12052 | combined  |       2.00 | open   | ["High amount (z=6.97)", "New device (NEVER_SE...)"] |
+----------+--------+-----------+------------+--------+-----------------------------------------------------+
```

Figure B.8 Alerts raised for the scripted persona transactions.

```
mysql> SELECT * FROM v_top_risky_users LIMIT 5;

+------------+-----------------+--------------------+--------------+-----------------+-----------------+
| account_id | full_name       | email              | total_alerts | avg_risk_score | max_risk_score |
+------------+-----------------+--------------------+--------------+-----------------+-----------------+
|          1 | Tammy Allison   | user1@example.com  |            6 |           1.17 |           2.00 |
|         20 | Anna Henderson  | user18@example.com |            1 |           3.00 |           3.00 |
|         46 | Kevin Pena      | user40@example.com |            1 |           3.00 |           3.00 |
|         77 | Rachael Pearson | user65@example.com |            1 |           3.00 |           3.00 |
|         94 | Gina Beard      | user79@example.com |            1 |           2.00 |           2.00 |
+------------+-----------------+--------------------+--------------+-----------------+-----------------+
```

Figure B.9 `v_top_risky_users`.

```
mysql> EXPLAIN SELECT COUNT(*) FROM transactions WHERE txn_time >= '2026-01-01' AND txn_time < '2026-01-08';

+----+-------------+--------------+------------+-------+-----------------------------------------------------------+-------------+---------+------+------+----------+--------------------------+
| id | select_type | table        | partitions | type  | possible_keys                                             | key         | key_len | ref  | rows | filtered | Extra                    |
+----+-------------+--------------+------------+-------+-----------------------------------------------------------+-------------+---------+------+------+----------+--------------------------+
|  1 | SIMPLE      | transactions | NULL       | range | idx_txn_merchant_time,idx_txn_time,idx_txn_account_time | idx_txn_time | 4       | NULL |  405 |   100.00 | Using where; Using index |
+----+-------------+--------------+------------+-------+-----------------------------------------------------------+-------------+---------+------+------+----------+--------------------------+
```

Figure B.10 `EXPLAIN` of a date-range query using `idx_txn_time`.

```
mysql> START TRANSACTION;
mysql> INSERT INTO transactions (account_id,merchant_id,device_id,location_id,amount,status) VALUES (1,1,1,1,95000,'success');
mysql> SELECT alert_id,txn_id,rule_name,risk_score,status,reasons FROM alerts WHERE txn_id=LAST_INSERT_ID();
mysql> ROLLBACK;

+----------+--------+-----------+------------+--------+-----------------------------------------------------+
| alert_id | txn_id | rule_name | risk_score | status | reasons                                             |
+----------+--------+-----------+------------+--------+-----------------------------------------------------+
|       50 |  12112 | combined  |       2.00 | open   | ["High amount (z=4.97)", "New device (93446f17...)"] |
+----------+--------+-----------+------------+--------+-----------------------------------------------------+
```

Figure B.11 Live trigger demonstration (insert, alert appears, rollback).

```
$ bash scripts/check.sh 11
ok    week 01  check.sql (2 assertions)
ok    week 02  check.sql (6 assertions)
ok    week 03  check.sql (8 assertions)
ok    week 04  check.sql (13 assertions)
ok    week 05  check.sql (5 assertions)
ok    week 06  check.sql (6 assertions)
ok    week 07  check.sql (6 assertions)
ok    week 08  check.sql (7 assertions)
ok    week 09  check.sql (20 assertions)
ok    week 10  check.sql (21 assertions)
ok    week 11  check.sql (8 assertions)
----
passed: 50   failed: 0
```

Figure B.12 SQL assertion suite, per-week results (2+6+8+13+5+6+6+7+20+21+8 = 102 assertions in 50 check files).

```
$ pytest -q
.........................                                [100%]
27 passed in 4.96s
```

Figure B.13 Python test suite.

Appendix C. Interactive Demonstration

`demo.html` is a standalone page that re-implements the six rules in JavaScript so the pipeline can be explained without a database connection. It is an explanatory aid; the authoritative behaviour is the SQL above. Screenshots below were captured from the page.

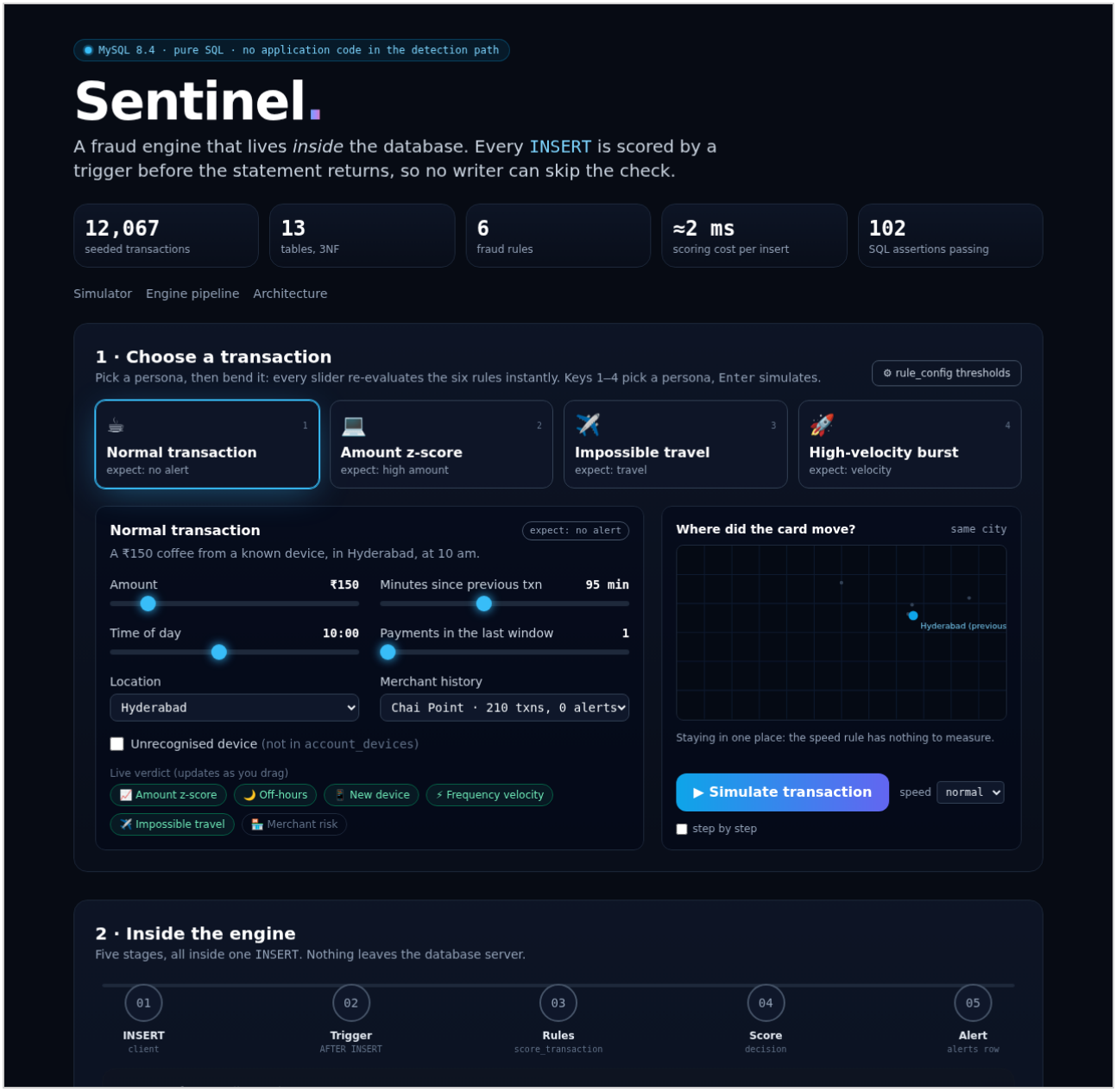


Figure C.1 Landing view.

MySQL 8.4 · pure SQL · no application code in the detection path

Sentinel.

A fraud engine that lives *inside* the database. Every `INSERT` is scored by a trigger before the statement returns, so no writer can skip the check.

12,067	13	6	≈2 ms	102
seeded transactions	tables, 3NF	fraud rules	scoring cost per insert	SQL assertions passing

Simulator · Engine pipeline · Architecture

1 · Choose a transaction

Pick a persona, then bend it: every slider re-evaluates the six rules instantly. Keys 1–4 pick a persona, Enter simulates.

⚙ rule_config thresholds

1	2	3	4
Normal transaction	**Amount z-score**	**Impossible travel**	**High-velocity burst**
expect: no alert	expect: high amount	expect: travel	expect: velocity

Amount z-score

expect: high amount

A sudden ₹95,000 electronics purchase on an account that averages about ₹1,180.

Amount ₹95,000 · Minutes since previous txn 245 min

Time of day 14:00 · Payments in the last window 1

Location: Hyderabad · Merchant history: Chai Point · 210 txns, 0 alerts

☐ Unrecognised device (not in account_devices)

Live verdict (updates as you drag)

📈 Amount z-score · FIRES · 🌙 Off-hours · 📱 New device · ⚡ Frequency velocity · ✈️ Impossible travel · 🏪 Merchant risk

Where did the card move?

same city



Staying in one place: the speed rule has nothing to measure.

↻ Simulate again · speed: normal

☐ step by step

2 · Inside the engine

Five stages, all inside one INSERT. Nothing leaves the database server.

01 INSERT (client) · 02 Trigger (AFTER INSERT) · 03 Rules (score_transaction) · 04 Score (decision) · 05 Alert (alerts row)

01 Ingestion · a client writes one row

```
INSERT INTO transactions
  (account_id, merchant_id, device_id, location_id, amount, status)
VALUES (7, 14, 12, 3, 95000.00, 'success');   -- Chai Point, Hyderabad
```

1 row stored · txn_id 12068 · amount ₹95,000 · city Hyderabad (stored; scoring has not run yet)

02 Trigger activation · the engine, not the client, makes the next call

```
CREATE TRIGGER trg_after_txn_insert
AFTER INSERT ON transactions FOR EACH ROW
BEGIN
  CALL score_transaction(NEW.txn_id);   -- NEW.txn_id = 12068
END;
```

⚡ trg_after_txn_insert · AFTER INSERT · FOR EACH ROW

03 Rule evaluation · score_transaction() checks six rules against rule_config

📈 Amount z-score · FIRES +1	🌙 Off-hours · clear	📱 New device · clear
n=65 · avg ₹2,623 · sd ₹11,659 → z = 7.92 (limit 3)	14:00 is 7.6% of history (limit 2.0%)	device found in account_devices
⚡ Frequency velocity · clear	✈️ Impossible travel · clear	🏪 Merchant risk · SKIPPED
1 txn in 5 min incl. this one (limit 5)	Hyderabad → Hyderabad: 0.0 km in 245 min = 0 km/h (limit 900)	Chai Point has 0 alerts; needs ≥ 3 before it is judged

04 Score and decision · each hit adds 1; any score above 0 raises an alert

score 1 > 0 → RAISE ALERT

Amount z-score. There is no combined threshold: each rule has its own, and one hit is enough.

```
INSERT INTO alerts (txn_id, rule_name, risk_score, reasons, status)
```

Figure C.2 Amount z-score persona.

Sentinel.

A fraud engine that lives *inside* the database. Every `INSERT` is scored by a trigger before the statement returns, so no writer can skip the check.

12,067	13	6	≈2 ms	102
seeded transactions	tables, 3NF	fraud rules	scoring cost per insert	SQL assertions passing

Simulator Engine pipeline Architecture

1 · Choose a transaction

Pick a persona, then bend it: every slider re-evaluates the six rules instantly. Keys 1–4 pick a persona, Enter simulates.

⚙ rule_config thresholds

1	2	3	4
Normal transaction expect: no alert	**Amount z-score** expect: high amount	**Impossible travel** expect: travel	**High-velocity burst** expect: velocity

Impossible travel — expect: travel

A payment in Tokyo 15 minutes after one in Hyderabad. About 6,300 km in a quarter of an hour.

Amount ₹1,800 — Minutes since previous txn 15 min

Time of day 11:00 — Payments in the last window 2

Location: Tokyo — Merchant history: Chai Point · 210 txns, 0 alerts

☐ Unrecognised device (not in account_devices)

Live verdict (updates as you drag)

📈 Amount z-score · 🌙 Off-hours · 📱 New device · ⚡ Frequency velocity · ✈️ Impossible travel · FIRES · 🏪 Merchant risk

Where did the card move? 6311 km



Implied speed 25,246 km/h vs limit 900

↻ Simulate again speed normal

☐ step by step

2 · Inside the engine

Five stages, all inside one INSERT. Nothing leaves the database server.

01 INSERT (client) → 02 Trigger (AFTER INSERT) → 03 Rules (score_transaction) → 04 Score (decision) → 05 Alert (alerts row)

01 Ingestion · a client writes one row

```
INSERT INTO transactions
  (account_id, merchant_id, device_id, location_id, amount, status)
VALUES (7, 14, 12, 9, 1800.00, 'success');   -- Chai Point, Tokyo
```

1 row stored txn_id 12068 amount ₹1,800 city Tokyo (stored; scoring has not run yet)

window: ● ● 2 payments in 5 min

02 Trigger activation · the engine, not the client, makes the next call

```
CREATE TRIGGER trg_after_txn_insert
AFTER INSERT ON transactions FOR EACH ROW
BEGIN
  CALL score_transaction(NEW.txn_id);   -- NEW.txn_id = 12068
END;
```

⚡ trg_after_txn_insert
AFTER INSERT · FOR EACH ROW

03 Rule evaluation · score_transaction() checks six rules against rule_config

📈 Amount z-score — clear n=65 · avg ₹1,190 · sd ₹718 → z = 0.85 (limit 3)	🌙 Off-hours — clear 11:00 is 8.0% of history (limit 2.0%)	📱 New device — clear device found in account_devices
⚡ Frequency velocity — clear 2 txns in 5 min incl. this one (limit 5)	✈️ Impossible travel — FIRES +1 Hyderabad → Tokyo: 6311.4 km in 15 min = 25,246 km/h (limit 900)	🏪 Merchant risk — SKIPPED Chai Point has 0 alerts; needs ≥ 3 before it is judged

04 Score and decision · each hit adds 1; any score above 0 raises an alert

score 1 > 0 → RAISE ALERT

Impossible travel. There is no combined threshold: each rule has its own, and one hit is enough.

Figure C.3 Impossible-travel persona.

Sentinel.

A fraud engine that lives *inside* the database. Every `INSERT` is scored by a trigger before the statement returns, so no writer can skip the check.

12,067	13	6	≈2 ms	102
seeded transactions	tables, 3NF	fraud rules	scoring cost per insert	SQL assertions passing

Simulator Engine pipeline Architecture

1 · Choose a transaction

Pick a persona, then bend it: every slider re-evaluates the six rules instantly. Keys 1–4 pick a persona, `Enter` simulates.

⚙ rule_config thresholds

1	2	3	4
Normal transaction	**Amount z-score**	**Impossible travel**	**High-velocity burst**
expect: no alert	expect: high amount	expect: travel	expect: velocity

High-velocity burst

expect: velocity

Six payments inside two minutes at different merchants. The sixth insert tips the rule.

Amount ₹905 — Minutes since previous txn 1 min

Time of day 16:00 — Payments in the last window 6

Location: Hyderabad — Merchant history: Chai Point · 210 txns, 0 alerts

☐ Unrecognised device (not in account_devices)

Live verdict (updates as you drag)

📈 Amount z-score · 🌙 Off-hours · 📱 New device · ⚡ Frequency velocity · FIRES · ✈️ Impossible travel · 🏪 Merchant risk

Where did the card move?

same city



Staying in one place: the speed rule has nothing to measure.

↻ Simulate again speed: normal

☐ step by step

2 · Inside the engine

Five stages, all inside one `INSERT`. Nothing leaves the database server.

01 INSERT (client) → 02 Trigger (AFTER INSERT) → 03 Rules (score_transaction) → 04 Score (decision) → 05 Alert (alerts row)

01 Ingestion · a client writes one row

```
INSERT INTO transactions
  (account_id, merchant_id, device_id, location_id, amount, status)
VALUES (7, 14, 12, 3, 905.00, 'success');   -- Chai Point, Hyderabad
```

1 row stored txn_id 12068 amount ₹905 city Hyderabad (stored; scoring has not run yet)

window: ● ● ● ● ● ● 6 payments in 5 min

02 Trigger activation · the engine, not the client, makes the next call

```
CREATE TRIGGER trg_after_txn_insert
AFTER INSERT ON transactions FOR EACH ROW
BEGIN
  CALL score_transaction(NEW.txn_id);   -- NEW.txn_id = 12068
END;
```

⚡ trg_after_txn_insert
AFTER INSERT · FOR EACH ROW

03 Rule evaluation · score_transaction() checks six rules against rule_config

📈 Amount z-score — clear n=65 · avg ₹1,176 · sd ₹715 → z = -0.38 (limit 3)	🌙 Off-hours — clear 16:00 is 7.1% of history (limit 2.0%)	📱 New device — clear device found in account_devices
⚡ Frequency velocity — FIRES +1 6 txns in 5 min incl. this one (limit 5)	✈️ Impossible travel — clear Hyderabad → Hyderabad: 0.0 km in 1 min = 0 km/h (limit 900)	🏪 Merchant risk — SKIPPED Chai Point has 0 alerts; needs ≥ 3 before it is judged

04 Score and decision · each hit adds 1; any score above 0 raises an alert

score 1 > 0 → RAISE ALERT

Frequency velocity. There is no combined threshold: each rule has its own, and one hit is enough.

Figure C.4 High-velocity persona.